

CSS Transitions

CSS Transitions

- CSS transitions allows you to change property values smoothly, over a given duration.
- [https://www.w3schools.com/css/css3_transitions.a
sp](https://www.w3schools.com/css/css3_transitions.asp)

- In this chapter you will learn about the following properties:
 - transition
 - transition-delay
 - transition-duration
 - transition-property
 - transition-timing-function

How to Use CSS Transitions?

- To create a transition effect, you must specify two things:
 - the CSS property you want to add an effect to
 - the duration of the effect
- **Note:** If the duration part is not specified, the transition will have no effect, because the default value is 0.

- The following example shows a 100px * 100px red <div> element. The <div> element has also specified a transition effect for the width property, with a duration of 2 seconds:
- The transition effect will start when the specified CSS property (width) changes value.

- Try it Yourself »

```
div:hover {  
  width: 300px;  
}
```

```
div {  
  width: 100px;  
  height: 100px;  
  background: red;  
  transition: width 2s;  
}
```

Change Several Property Values

- The following example adds a transition effect for both the width and height property, with a duration of 2 seconds for the width and 4 seconds for the height:

```
div {  
  transition: width 2s, height 4s;  
}
```

- [Try it Yourself »](#)

Specify the Speed Curve of the Transition

- The **transition-timing-function** property specifies the speed curve of the transition effect.
- The transition-timing-function property can have the following values:
 - **ease** - specifies a transition effect with a slow start, then fast, then end slowly (this is default)
 - **linear** - specifies a transition effect with the same speed from start to end
 - **ease-in** - specifies a transition effect with a slow start
 - **ease-out** - specifies a transition effect with a slow end
 - **ease-in-out** - specifies a transition effect with a slow start and end
 - **cubic-bezier(n,n,n,n)** - lets you define your own values in a cubic-bezier function

- The following example shows some of the different speed curves that can be used

```
#div1 {transition-timing-function: linear;}  
#div2 {transition-timing-function: ease;}  
#div3 {transition-timing-function: ease-in;}  
#div4 {transition-timing-function: ease-out;}  
#div5 {transition-timing-function: ease-in-out;}
```

- [Try it Yourself »](#)

Delay the Transition Effect

- The **transition-delay** property specifies a delay (in seconds) for the transition effect.
- The following example has a 1 second delay before starting

```
div {  
    transition-delay: 1s;  
}
```

- [Try it Yourself »](#)

Transition + Transformation

- The following example adds a transition effect to the transformation

```
div {  
  transition: width 2s, height 2s, transform 2s;  
}
```

- [Try it Yourself »](#)

More Transition Examples

- The CSS transition properties can be specified one by one, like this:

```
div {  
  transition-property: width;  
  transition-duration: 2s;  
  transition-timing-function: linear;  
  transition-delay: 1s;  
}
```

- [Try it Yourself »](#)

- or by using the shorthand property **transition**:

```
div {  
    transition: width 2s linear 1s;  
}
```

- [Try it Yourself »](#)

CSS Animations

CSS Animations

- CSS allows animation of HTML elements without using JavaScript or Flash!
- https://www.w3schools.com/css/css3_animations.asp

- In this chapter you will learn about the following properties:
 - @keyframes
 - animation-name
 - animation-duration
 - animation-delay
 - animation-iteration-count
 - animation-direction
 - animation-timing-function
 - animation-fill-mode
 - animation

What are CSS Animations?

- An animation lets an element gradually change from one style to another.
- You can change as many CSS properties you want, as many times you want.
- To use CSS animation, you must first specify some keyframes for the animation.
- Keyframes hold what styles the element will have at certain times.

The @keyframes Rule

- When you specify CSS styles inside the **@keyframes** rule, the animation will gradually change from the current style to the new style at certain times.
- To get an animation to work, you must bind the animation to an element.

- The following example binds the "example" animation to the <div> element. The animation will last for 4 seconds, and it will gradually change the background-color of the <div> element from "red" to "yellow":

```
/* The animation code */
@keyframes example {
  from {background-color: red;}
  to {background-color: yellow;}
}

/* The element to apply the animation to */
div {
  width: 100px;
  height: 100px;
  background-color: red;
  animation-name: example;
  animation-duration: 4s;
}
```

- [Try it Yourself »](#)

- **Note:** The **animation-duration** property defines how long time an animation should take to complete. If the animation-duration property is not specified, no animation will occur, because the default value is 0s (0 seconds).
- In the example above we have specified when the style will change by using the keywords "from" and "to" (which represents 0% (start) and 100% (complete)).
- It is also possible to use percent. By using percent, you can add as many style changes as you like.

- The following example will change the background-color of the <div> element when the animation is 25% complete, 50% complete, and again when the animation is 100% complete:

```
/* The animation code */
@keyframes example {
  0%   {background-color: red;}
  25%  {background-color: yellow;}
  50%  {background-color: blue;}
  100% {background-color: green;}
}

/* The element to apply the animation to */
div {
  width: 100px;
  height: 100px;
  background-color: red;
  animation-name: example;
  animation-duration: 4s;
}
```

- Try it Yourself »

- The following example will change both the background-color and the position of the <div> element when the animation is 25% complete, 50% complete, and again when the animation is 100% complete:

- [Try it Yourself »](#)

```
/* The animation code */
@keyframes example {
  0%   {background-color:red; left:0px; top:0px;}
  25%  {background-color:yellow; left:200px; top:0px;}
  50%  {background-color:blue; left:200px; top:200px;}
  75%  {background-color:green; left:0px; top:200px;}
  100% {background-color:red; left:0px; top:0px;}
}

/* The element to apply the animation to */
div {
  width: 100px;
  height: 100px;
  position: relative;
  background-color: red;
  animation-name: example;
  animation-duration: 4s;
}
```

Delay an Animation

- The **animation-delay** property specifies a delay for the start of an animation.
- The following example has a 2 seconds delay before starting the animation:

```
div {  
  width: 100px;  
  height: 100px;  
  position: relative;  
  background-color: red;  
  animation-name: example;  
  animation-duration: 4s;  
  animation-delay: 2s;  
}
```

- Try it Yourself »

- Negative values are also allowed. If using negative values, the animation will start as if it had already been playing for N seconds.
- In the following example, the animation will start as if it had already been playing for 2 seconds:

```
div {  
  width: 100px;  
  height: 100px;  
  position: relative;  
  background-color: red;  
  animation-name: example;  
  animation-duration: 4s;  
  animation-delay: -2s;  
}
```

- [Try it Yourself »](#)

Set How Many Times an Animation Should Run

- The **animation-iteration-count** property specifies the number of times an animation should run.
- The following example will run the animation 3 times before it stops.

- Try it Yourself »

```
div {  
  width: 100px;  
  height: 100px;  
  position: relative;  
  background-color: red;  
  animation-name: example;  
  animation-duration: 4s;  
  animation-iteration-count: 3;  
}
```


- The following example uses the value "infinite" to make the animation continue for ever:

```
div {  
  width: 100px;  
  height: 100px;  
  position: relative;  
  background-color: red;  
  animation-name: example;  
  animation-duration: 4s;  
  animation-iteration-count: infinite;  
}
```

- Try it Yourself »

Run Animation in Reverse Direction or Alternate Cycles

- The **animation-direction** property specifies whether an animation should be played forwards, backwards or in alternate cycles.
- The animation-direction property can have the following values:
 - **normal** - The animation is played as normal (forwards). This is default
 - **reverse** - The animation is played in reverse direction (backwards)
 - **alternate** - The animation is played forwards first, then backwards
 - **alternate-reverse** - The animation is played backwards first, then forwards

- The following example will run the animation in reverse direction (backwards):

- Try it Yourself »

```
div {  
  width: 100px;  
  height: 100px;  
  position: relative;  
  background-color: red;  
  animation-name: example;  
  animation-duration: 4s;  
  animation-direction: reverse;  
}
```

- The following example uses the value "alternate" to make the animation run forwards first, then backwards:

```
div {  
  width: 100px;  
  height: 100px;  
  position: relative;  
  background-color: red;  
  animation-name: example;  
  animation-duration: 4s;  
  animation-iteration-count: 2;  
  animation-direction: alternate;  
}
```

- [Try it Yourself »](#)

- The following example uses the value "alternate-reverse" to make the animation run backwards first, then forwards:

```
div {  
  width: 100px;  
  height: 100px;  
  position: relative;  
  background-color: red;  
  animation-name: example;  
  animation-duration: 4s;  
  animation-iteration-count: 2;  
  animation-direction: alternate-reverse;  
}
```

- [Try it Yourself »](#)

Specify the Speed Curve of the Animation

- The **animation-timing-function** property specifies the speed curve of the animation.
- The animation-timing-function property can have the following values:
 - **ease** - Specifies an animation with a slow start, then fast, then end slowly (this is default)
 - **linear** - Specifies an animation with the same speed from start to end
 - **ease-in** - Specifies an animation with a slow start
 - **ease-out** - Specifies an animation with a slow end
 - **ease-in-out** - Specifies an animation with a slow start and end
 - **cubic-bezier(n,n,n,n)** - Lets you define your own values in a cubic-bezier function

- The following example shows some of the different speed curves that can be used:

```
#div1 {animation-timing-function: linear;}  
#div2 {animation-timing-function: ease;}  
#div3 {animation-timing-function: ease-in;}  
#div4 {animation-timing-function: ease-out;}  
#div5 {animation-timing-function: ease-in-out;}
```

Specify the fill-mode For an Animation

- CSS animations do not affect an element before the first keyframe is played or after the last keyframe is played. The animation-fill-mode property can override this behavior.

- The **animation-fill-mode** property specifies a style for the target element when the animation is not playing (before it starts, after it ends, or both).
- The animation-fill-mode property can have the following values:
 - **none** - Default value. Animation will not apply any styles to the element before or after it is executing
 - **forwards** - The element will retain the style values that is set by the last keyframe (depends on animation-direction and animation-iteration-count)
 - **backwards** - The element will get the style values that is set by the first keyframe (depends on animation-direction), and retain this during the animation-delay period
 - **both** - The animation will follow the rules for both forwards and backwards, extending the animation properties in both directions

- The following example lets the <div> element retain the style values from the last keyframe when the animation ends:

```
div {  
  width: 100px;  
  height: 100px;  
  background: red;  
  position: relative;  
  animation-name: example;  
  animation-duration: 3s;  
  animation-fill-mode: forwards;  
}
```

- [Try it Yourself »](#)

- The following example lets the <div> element get the style values set by the first keyframe before the animation starts (during the animation-delay period):

```
div {  
  width: 100px;  
  height: 100px;  
  background: red;  
  position: relative;  
  animation-name: example;  
  animation-duration: 3s;  
  animation-delay: 2s;  
  animation-fill-mode: backwards;  
}
```

- [Try it Yourself »](#)

- The following example lets the <div> element get the style values set by the first keyframe before the animation starts, and retain the style values from the last keyframe when the animation ends:

```
div {  
  width: 100px;  
  height: 100px;  
  background: red;  
  position: relative;  
  animation-name: example;  
  animation-duration: 3s;  
  animation-delay: 2s;  
  animation-fill-mode: both;  
}
```

- [Try it Yourself »](#)

Animation Shorthand Property

- The example below uses six of the animation properties.

```
div {  
  animation-name: example;  
  animation-duration: 5s;  
  animation-timing-function: linear;  
  animation-delay: 2s;  
  animation-iteration-count: infinite;  
  animation-direction: alternate;  
}
```

- The same animation effect as above can be achieved by using the shorthand animation property:

```
div {  
  animation: example 5s linear 2s infinite alternate;  
}
```

Exercise 16

- <https://reurl.cc/2lzmlO>
- Practice simple CSS transitions. Please create a web page containing two squares. The shapes of squares change when the mouse hovers on them. Please upload the web page to the webspace and upload the link to the portal.

- Mouse hover square 1: the square becomes longer.
- Mouse hover square 2: the square becomes larger.

